Option 1: The "Classic Hotel" Briefs: based on Tut Week 3

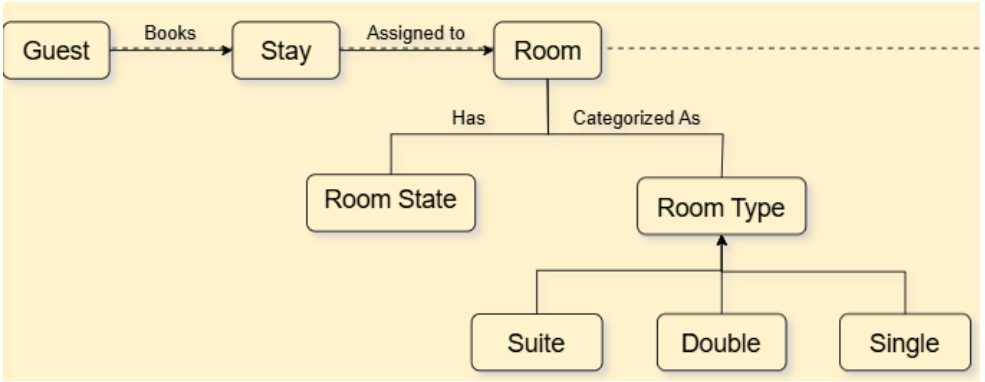
Group 1: The Domain & Data Architects

Scenario: You are consulting for the Midland Hotel. The hotel wants a new system, but the previous developers started building databases without understanding how the hotel works.

Presentation Prompts:

1. **Product vs. Domain:** Look at the hotel receptionist. Is the receptionist part of the *Product* or part of the *Domain*? Explain your reasoning to the class. *(Refers to Q1)*
   => Based on lectures and pre-readings, the receptionist is part of the Domain because Domain represents real-world business environment. In detail, the receptionist exists, manage physical keys, and greets guests regardless of whether the software exists.

2. **Domain Modeling:** The current domain model only has Guest, Stay, Room State, and Room. However, the hotel has different room types (Single, Double, Suite) and needs to track them. Present an updated Domain Model diagram that includes these extensions.*(Refers to Q5)*

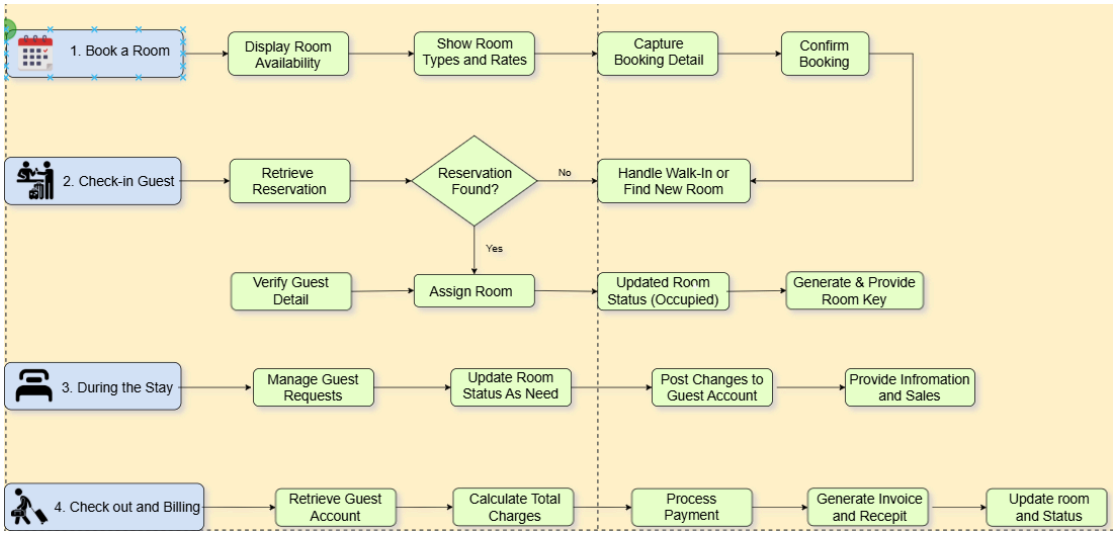


3. **Tacit Requirements:** What is an example of a "tacit requirement" a hotel manager might have but forget to tell you? How do you validate it? *(Refers to Q4)*
   => A tactic requirement is unspoken rule stakeholders assume you already know, like forgetting to mention that "rooms cannot be assigned to arriving guests until Housekeeping flags them as 'Cleaned' and 'Inspected' after checkout." If omitted, developers risk building a system that allows check-ins to dirty rooms. We validate these hidden rules using two techniques: High-Level Workflows (Activity Diagrams) to visually walk the manager through the room's lifecycle to confirm the steps, and "Variants & Problems" Task Specifications by intentionally writing a problem scenario (e.g., selected room is dirty) to force the manager to verbalize how the system should handle the exception

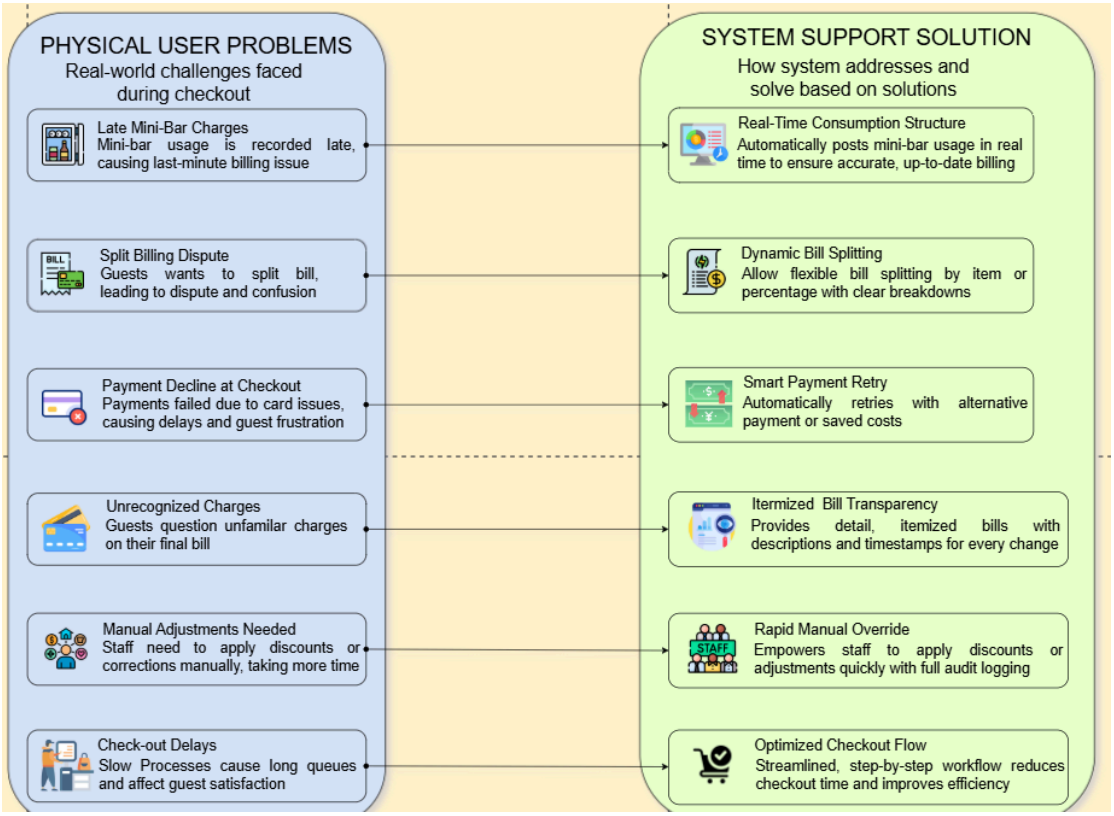
Group 2: The Task & Support Engineers

Scenario: The Midland Hotel's current system is terrible for the front-desk staff. You need to map
out their actual workflows using the "Task & Support" methodology. **Presentation Prompts:**

1. **Workflow:** Identify the typical high-level tasks a Hotel Information System must support. Visualize this as a workflow diagram. *(Refers to Q3)*
   => The core operations into **4 high-level tasks** using the Task & Support methodology for the front-end desk:
   - **Book Room**: The system supports checking real-time room availability and creating bookings.
   - **Check-in Guest**: The system supports matching booking IDs, verifying that room states are 'Cleaned', and updating room availability to 'Occupied'.
   - **During the Stay**: The system supports logging dynamic expenses like room service or laundry directly into the active stay record.
   - **Check-out & Billing**: The system supports generating a single consolidated bill, processing payments, and automatically shifting the room state to 'Dirty' for housekeeping."



2. **Task & Support (Check-Out):** Present a formal "Task & Support" table for the **Check Out** task. Be sure to include sub-tasks, physical problems the user faces, and your proposed system solutions. (Refers to Q2).



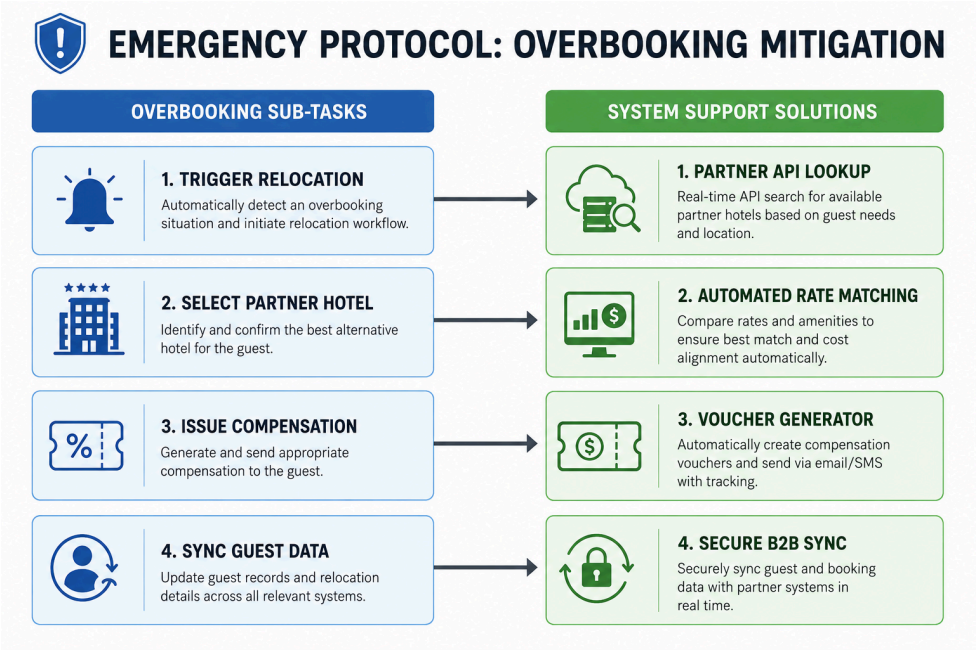
3. **Splitting Tasks (Check-In):** The Check-In task handles two situations:
   A) Guest booked in advance,
   B) Guest walks in without booking. Should this be one task or split into two? Argue which approach is better for the system architecture. *(Refers to Q6)*
   => Split into two tasks because following reason:
   - Clean Code: Merging creates messy `if/else` logic; splitting keeps them modular so changing walk-in rules won't break the stable booking pipeline.
   - Read vs. Write: Pre-booked is a quick Query (Read) updating a `Booking ID`; walk-in is a Command (Write) initializing a new record, dynamic pricing, and real-time room allocation.
   - DB Concurrency: Walk-ins require strict database row-locking to prevent two clerks from booking the same vacant room simultaneously; pre-booked flows bypass this as rooms are already secured.
   - Tailored UI/UX: Pre-booked needs a rapid 30-second lookup screen; walk-in needs an extensive data-entry form with real-time room filters.

Group 3: The Edge-Case Analysts

Scenario: Everything goes wrong eventually. Your job is to design a system that handles business exceptions gracefully without crashing into the user's workflow.

Presentation Prompts:

1. **The Overbooking Problem:** The hotel explicitly overbooks rooms by 5% because guests often cancel. However, today, everyone showed up. Extend the "Check-In" task in a Task & Support table to handle this emergency (e.g., allocating a guest to a partner hotel). *(Refers to Q7)*



2. **Goal vs. Design:** If the hotel manager says, *"I want a red warning box to pop up when the hotel is overbooked,"* where does this fall on the Goal-Design Scale? How would you push this back up to the Domain level?

Level	Statement Example	Why it's there
Domain Level — Why we exist	*"Ensure every guest has a safe and suitable place to stay, even when we overbook."*	Defines the business purpose and desired outcome. Guides priorities and aligns everyone on what success looks like.
Task Level — What we need to do	*"Relocate the overbooked guest to a partner hotel and provide appropriate compensation."*	Breaks down the goal into key business tasks. Ensures all necessary work is identified and understood.
Design Level — How we build it	*"Implement an API integration with partner systems to search availability and book rooms in real time."*	Defines the technical solution and implementation approach. Addresses system behavior, constraints, and integration details.

Option 2: Modern Case Studies

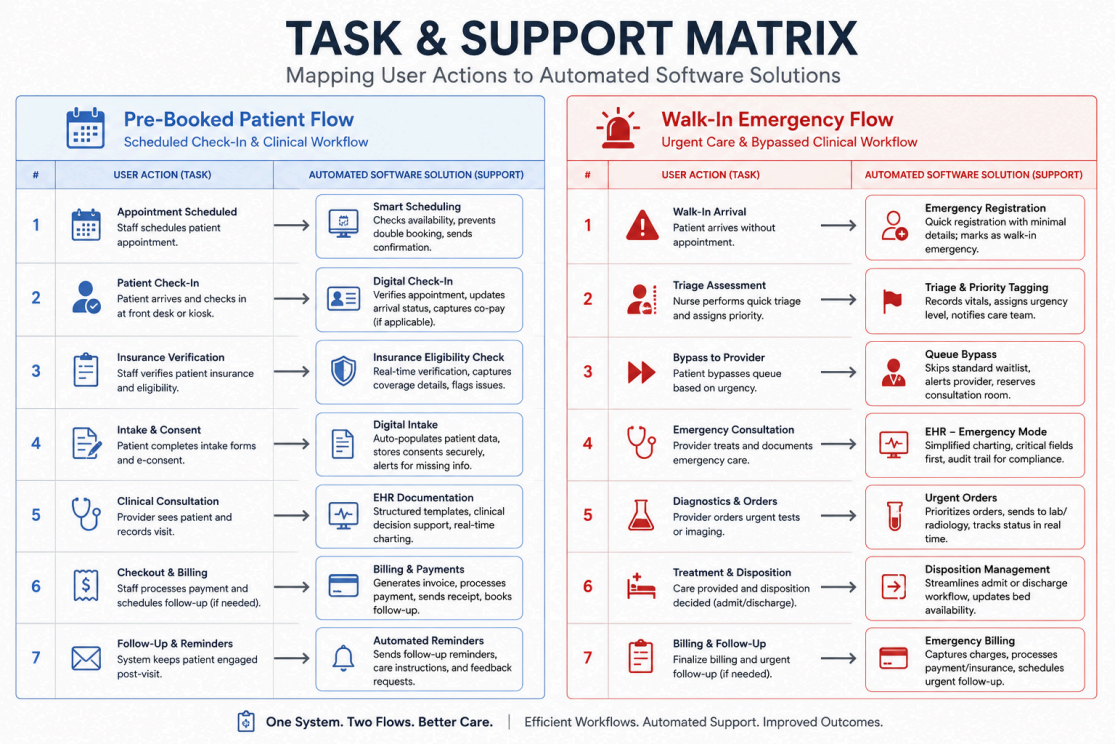
Group 4: Case Study - "QuickMed" Walk-in Clinic

Scenario: QuickMed is a modern urgent-care clinic. Patients can either book online or walk in off the street bleeding. They currently use paper clipboards.

Presentation Prompts:

1. **Domain vs Product:** Is the Triage Nurse taking the patient's blood pressure part of the Domain or the Product?
=> It is part of the domain because

2. **Task & Support:** Create a Task & Support table for the "Patient Intake/Check-In" task. How do the sub-tasks differ between a pre-booked patient and a walk-in emergency?

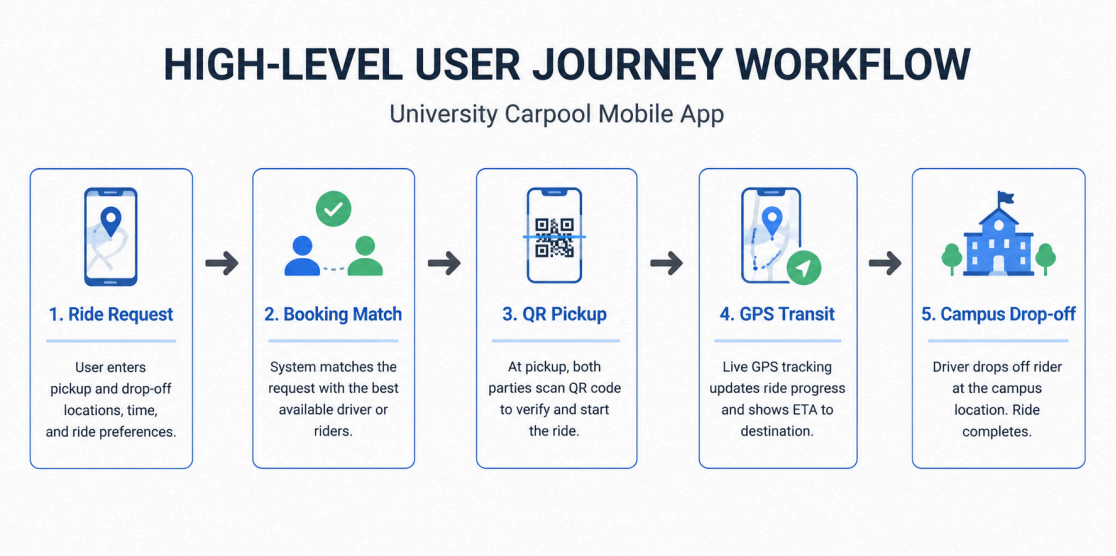


3. **Tacit Requirements:** What is a tacit requirement the clinic doctors might expect regarding patient history that they won't explicitly ask for
=> While doctors explicitly ask to "view history," they tacitly expect the software to instantly surface critical risks—like active blood thinners or severe drug allergies—filtering out irrelevant past records during an active emergency.

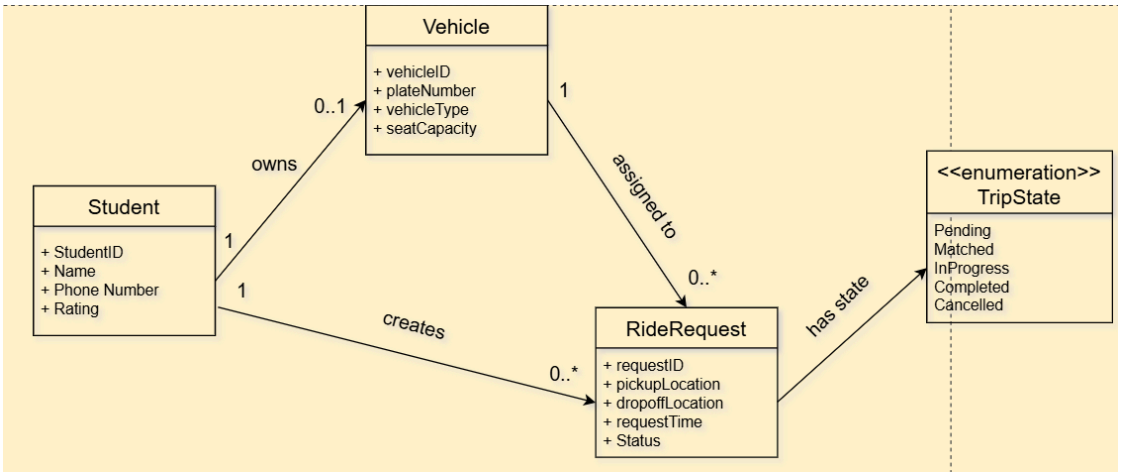
Group 5: Case Study - "CampusRide" University Carpool App

Scenario: The university is launching an app where students can offer empty seats in their cars to other students commuting to campus. **Presentation Prompts:**

1. **High-Level Workflow:** Map out the high-level workflow from the moment a student decides they need a ride, to the moment they are dropped off.



2. **Domain Modeling:** Draw a simple Domain Model. Be sure to separate the Student, the Vehicle, the Ride Request, and the Trip State (e.g., pending, in-progress, completed).



3. **Edge Case (Similar to Overbooking):** A driver accepts a ride but cancels 5 minutes before pickup. Create a Task description for how the system and the stranded student handle this exception.